

M18 Project Handbook

How to build portfolio projects without copying a hidden final answer

Evidence chain

Requirement -> source contract -> artifact -> validation -> failure/change -> recovery -> fresh retry -> explanation. If one link is missing, the project is not defensible.

Grain before tools

Write what one source row/event and one trusted output row mean before joins, models or dashboards. Wrong grain can create plausible but false totals.

Raw vs trusted

Raw preserves what arrived and when. Trusted output is derived only after explicit contracts/quality. Do not clean raw in place.

Idempotency and state

Same logical input should not multiply trusted state. Persist state/checkpoint only after durable success; test exact rerun and one changed/late input.

Quality as publish decision

Critical defects block trusted publish or quarantine records/batches explicitly. Warnings are not an excuse to hide bad data.

Failure evidence

Freeze the first failing output/log before repair. State root cause, smallest repair, post-repair validation and regression check.

Reproducibility

A reviewer should be able to start from a clean copy using documented dependencies/config. Hidden local state is a defect.

Runtime truth

Separate executed, simulated/static and not-executed claims. Never describe a cloud/container/distributed run you did not perform.

Portfolio/publication

Sanitize secrets/PII, make README useful, include architecture/runbook/data dictionary/evidence index, and keep synthetic/sample data clearly labeled.

Technical defense

Explain problem, source grain, trusted grain, architecture, key controls, rerun/backfill, one failure/recovery, security, cost/scale and one rejected alternative without reading a script.

What the checker can and cannot prove

- It can prove an evidence file exists and required fields are populated.
- It can prove source/profile scripts run against bundled fixtures.
- It can prove package hashes/ZIP integrity during course QA.
- It cannot prove your SQL/model/architecture is semantically correct.
- It cannot prove a live tool ran unless you save genuine observed runtime evidence.
- It cannot replace protected Review or human/AI-assisted technical review after your attempt.

Help ladder

- 1) Restate the phase objective and output grain.
- 2) Inspect the exact source/artifact contract and smallest failing evidence.
- 3) Reopen only the relevant prior module/Quick Reference.
- 4) After a saved attempt, open only the matching protected phase rubric.
- 5) Repair one cause, close Review, run a changed retry.
- 6) If a runtime/tool fails, separate environment repair from business-logic repair.

Suggested evidence folder

`evidence//attempt.json` is the canonical phase evidence record. Put screenshots/logs/output files beside it or reference their relative paths. If you retry, use attempt-02 or a fresh-retry file; never overwrite attempt-01.